
Expecting the Unexpected

(Errors in Python)

Unit 3

Concepts

- Raising an exception
- The effects of an exception
- Handling exceptions
- The exception hierarchy
- Defining our own exceptions

Errors in a Python Program

- Compile Time Errors: syntactical errors found in code
- Runtime Errors: Runtime error is not detected by compiler but detected by PVM.
- Logical Errors: The programmer might be using wrong formula or the design of the program itself is wrong.

Raising an exception

In Python, there are several built-in Python exceptions that can be raised when an error occurs during the execution of a program.

- **SyntaxError:** This exception is raised when the interpreter encounters a syntax error in the code, such as a misspelled keyword, a missing colon, or an unbalanced parenthesis.
- **TypeError:** This exception is raised when an operation or function is applied to an object of the wrong type, such as adding a string to an integer.
- **NameError:** This exception is raised when a variable or function name is not found in the current scope.
- **IndexError:** This exception is raised when an index is out of range for a list, tuple, or other sequence types.
- **ZeroDivisionError:** This exception is raised when an attempt is made to divide a number by zero.
- **ImportError:** This exception is raised when an import statement fails to find or load a module.

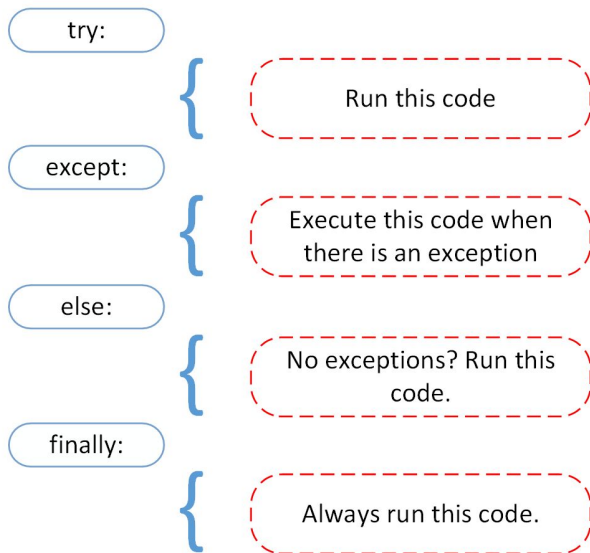
The effects of an exception

Errors are problems in a program due to which the program will stop the execution.

On the other hand, exceptions are raised when some internal events occur which change the normal flow of the program.

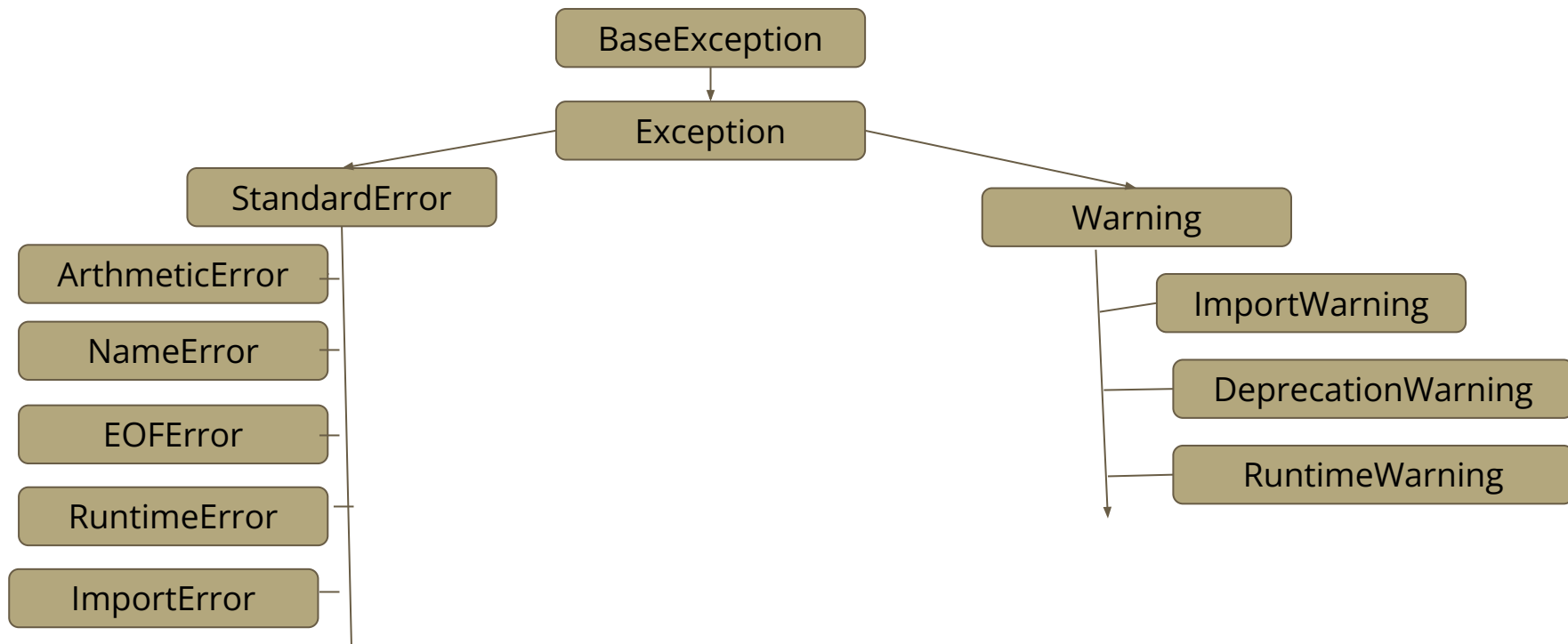
Handling exceptions

Syntax:



```
try:  
    print(x)  
except NameError:  
    print("undefined variable")  
else:  
    print("something went wrong")  
finally:  
    print("Thank you for coding!")
```

The exception hierarchy



Defining our own exceptions

```
class FileProcessingError(Exception):
    def __init__(self, message, filename, lineno):
        super().__init__(message)
        self.filename = filename
        self.lineno = lineno

    def __str__(self):
        return f"{self.message} in {self.filename} at line {self.lineno}"

try:
    raise FileProcessingError("Syntax error", "example.txt", 13)
except FileProcessingError as e:
    print(f"Caught an error: {e}")
```